FACULTY OF COMPUTING

SEMESTER 2 2025/2026

SECP3843 – SPECIAL TOPIC IN DATA ENGINEERING

SECTION 2

TUTORIAL 2:

APACHE SPARK

LECTURER: DR. ARYATI BINTI BAKRI

GROUP 2

STUDENT NAME	MATRIC NO
CHAU YING JIA	A23CS0213
LAU YEE WEN	A23CS0099
POH LOK YEE	A23CS0262

1.0 Business Understanding

1.1 Background

This project started from the tutorial “Creating a Simple ETL Pipeline Using Apache Spark” created by João Pedro. The project aims to create a data pipeline using Apache Spark to extract raw educational data from the Brazilian government’s open data website, transform it into a structured format, and load it into a PostgreSQL database for analysis and visualisation. The data are taken from the Brazilian National Basic Education Census (Censo Escolar), which was carried out from 2010 to 2021 by the National Institute of Education Research and Education (INEP). All basic education schools in Brazil are counted in the census from different aspects, including structure, race, geography, and economy.

1.2 Problem Statement

The original census data was published in large CSV files, containing a total of 12 files of about 180MB each, with a total size of around 2.2GB. Because of the large amount of data, the data could not be analysed easily using traditional tools such as Pandas, as these tools could become very slow or crash when used with such a large dataset. The main challenge of this project is the processing, organization, and analysis of this huge amount of data in an efficient way, so that relevant questions about educational trends in Brazil between 2010 and 2021 can be answered.

1.3 Business Objectives

The project is to provide answers to issues like the number of students who receive basic education in 2021, whether the total enrolment of the students changes over the years or not, and which States or Cities in Brazil have the highest enrolment of the students in 2021. To efficiently answer these questions, the data should be stored in a star schema in the SQL database. This schema is designed for doing data aggregations, not to remove data redundancy.

1.4 Proposed Solution

A 4-step ETL process is implemented using Apache Spark as the solution. First, CSV files are extracted and downloaded from the Brazilian Open Data Portal. Secondly, the CSV files are converted into Parquet format, which reduces the file size to around one-fifth of the original size and allows much faster query time. Thirdly, the data is transformed and restructured to create a star schema with dimension tables and fact tables. Finally, the structured data is loaded into a PostgreSQL database and visualised through Adminer and Metabase dashboards. Apache Spark is selected because a lazy evaluation model is supported, which means that data is only loaded into memory when it is required. Therefore, it is suitable for datasets of this size.

Since this project focuses on analytical queries involving aggregation operations such as SUM, COUNT and GROUP BY, rather than on minimizing disk usage or data redundancy, a star schema was adopted instead of the normalized relational model. In a star schema, all the metrics are stored in a fact table in the center, with dimension tables providing information about the environment in which each fact is stored. This design will allow faster aggregation queries and will facilitate the construction of dashboards that are directly related to the business objectives of this project.

2.0 Data Understanding

The goal of this phase is to have a clear insight of the dataset used. This dataset is explored through its origin, internal structure and the logic used to organize the raw data into a relational format.

2.1 Data Exploration

Based on Figure 2.1, the dataset used in this tutorial comprises 12 CSV files provided by Brazil's National Institute for Educational Studies and Research(INEP). This data covers the Brazilian Basic Education Census(*Censo Escolar da Educação Básica*) from 2010 to 2021. Each file is approximately 180 MB - 440 MB in size.

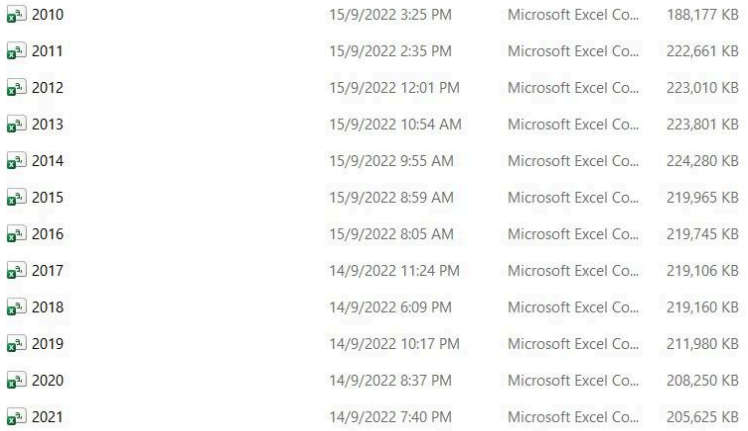

Figure 2.1 11 CSV files

Key characteristics identified during initial exploration using Apache Spark are summarised below:

Characteristic	Detail
Number of files	12 CSV files (one per year, 2010–2021)
Total size	Approximately 2.79million rows, 2.2GB
Columns per file	370 columns
Delimiter	Semicolon (**;**)
Encoding	Used **latin1** for Portuguese characters
Data source	INEP — Brazilian Government Open Data Portal

Table 2.1 Characteristics of raw data

Key insights after exploration of data:

- Enrollment numbers show a declining trend from approximately 51.5 million in 2010 to 46.7 million in 2020, showing the demographic change in Brazil's school-age population.
- **São Paulo** was recorded as the highest enrollment figures among all cities across all years.
- All columns were initially loaded as strings due to the usage of **inferSchema=false** instead of **inferSchema=true**. It is used for better and faster loading performance to reduce the waiting time. However, it requires manual type casting during the transformation phase.

Example of data types Identified from the raw data are:

Column Type	Examples	Category
Integer	QT_MAT_BAS, QT_DOC_BAS, NU_ANO_CENSO	Counts and metrics
String/Text	NO_UF, SG_UF, NO_MUNICIPIO	Location descriptors
Binary (0/1)	IN_BANHEIRO, IN_INTERNET, IN_AGUA_POTAVEL	Facility indicators

Table 2.2 Data types examples of the 370 columns

Issues identified when dealing with the raw data:

- All columns stored as strings because **inferSchema=false** was used. It requires manual casting to integers later in the transformation phase.
- Required code adjustment as file encoding has to use **latin1** but Spark 4.1.1 only accepts **iso-8859-1.**
- Wildcard ***.csv** didn't work on Windows with Spark, it required a glob library instead.
- Missing data, some years had missing or incomplete data for certain facility indicator columns.

2.2 Relational Data Model Design

Section 2.2 explains the process of analyzing and transforming the raw data into a structured Relational Data Model. Phase 1 identifies that the original flat CSV file suffers from data redundancy and lacks a unique primary key. Phase 2 resolves these anomalies by decomposing the unnormalized dataset into six distinct related entities. Ultimately, this normalized design eliminates data redundancy to optimize storage efficiency. In the end, these phases serve as a reliable blueprint before building the final star schema.

2.2.1 Phase 1 of Data Model Design

Since the raw data exists as a single flat CSV file with 370 columns, a conceptual Relational Data Model was constructed. It is to understand the entities and relationships hidden within the flat structure as a foundation for designing the targeted Star Schema taught in this tutorial.

Since there is only one source table for each year, the ER diagram for the raw data should only show one entity with grouped attributes . The ER diagram is as shown in Figure 2.2.

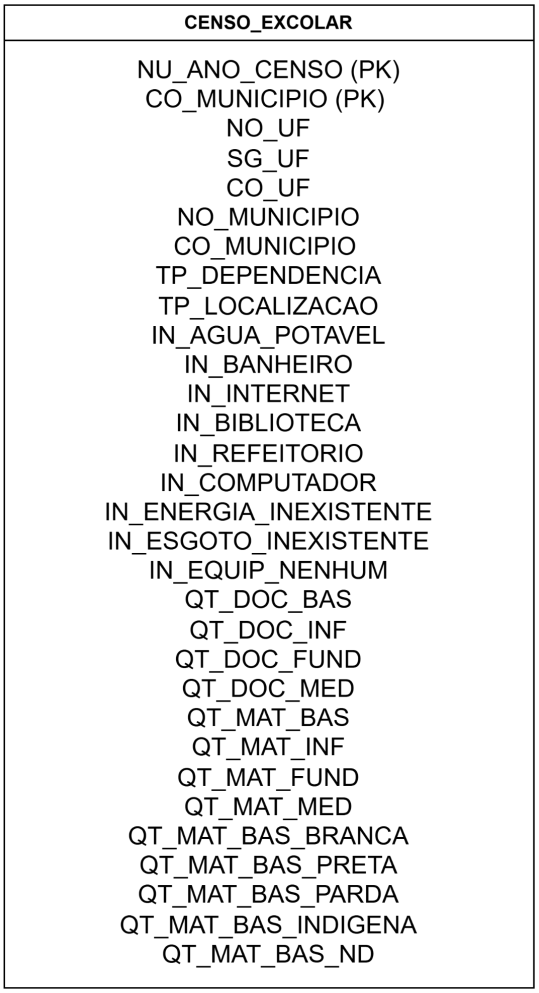

Figure 2.2 ERD of the raw data

The raw data does not have a specific single primary key column. Therefore, the **NU_ANO_CENSO** (year) and **CO_MUNICIPIO** (municipality code) are combined together to uniquely identify each record of one school in one year.

Next, since all data is in one flat table, there are no relationships between tables. Everything is stored together in one file including all the locations, facilities, number of teachers and number of enrollment yearly all in one row.

Furthermore, there are actually redundancy problems in this raw dataset since the data is not normalized. For example, the state name (**NO_UF = "São Paulo")** and its abbreviation name **(SG_UF=”SP”)** are repeated for every single school in **São Paulo** every single year. This means the school in the same location is duplicated millions of times. This redundancy wastes storage and will slow down queries.

2.2.2 Phase 2 of Data Model Design

After exploring the original dataset, we could conclude that the raw data is actually denormalized with data redundancy. In order to solve this problem, we try to proceed to the next phase which is to propose a normalized structure by separating the raw dataset into multiple related entities. Based on Table 2.3, there are six main entities identified: Location, School, Facilities, Teachers, Race, and Enrollment.

Entity	Description
Location	Geographic attributes of the school (eg. state, municipality)
School Type	School information
Facilities	Infrastructure indicators (eg. water, internet, library, etc.)
Teachers	Count of teachers by education level
Race	Race and ethnicity of students
Enrollments	Student enrollment counts by level and race/ethnicity

Table 2.3 Entities identified from the data

Figure 2.3 shows a normalized ER Diagram. The central entity is School which acts as the main reference point. Other entities are linked to it either directly or indirectly to ensure that repeated information such as location and the facility indicator is stored only once. As a result, this relational model design improves data consistency and storage efficiency of the database.

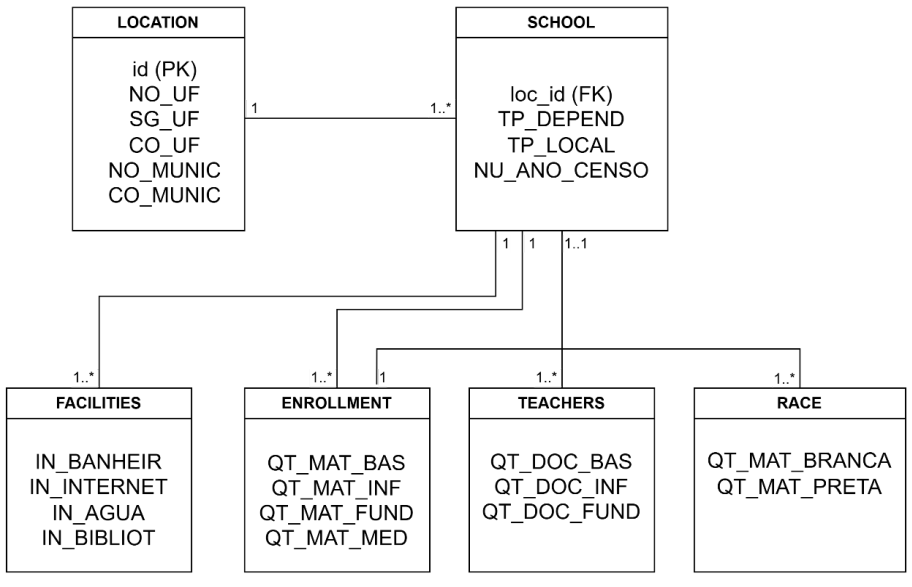

Figure 2.3 Normalized ER Diagram

Figure 2.4 shows the relationships between all the entities. There are 5 relationships between them.

Entities	Relationships
LOCATION → SCHOOL	One-to-Many (1 : 0..*)
SCHOOL → FACILITIES	One-to-Many (1 : 1..*)
SCHOOL → ENROLLMENT	One-to-Many (1 : 1..*)
SCHOOL → TEACHERS	One-to-Many (0..1 : 1..*)
ENROLLMENT → RACE	One-to-Many (1 : 1..*)

Figure 2.4 Relationships between Entities

Overall, the normalized ER diagram restructures the dataset into well-defined entities with clear relationships. The separation of Location, School and the other attributes eliminates redundancy and ensures that each data element is stored in its most appropriate entity. This design improves data integrity and supports efficient querying and scalability for future system expansion.

2.2.3 Normalization Analysis

Normalization is the process of organizing or splitting data into smaller tables to reduce redundancy, eliminate unwanted characteristics and maintain data integrity. It is done by following a set of rules known as normal forms.

First Normal Form (1NF) - The first normal form is satisfied in this relational data model as all the attribute values in the dataset are atomic. Each cell in the data.csv contains a single, indivisible value. For example, **QT_MAT_BAS** contains a single integer count and SG_UF contains only a single two-letter state abbreviation. There are no repeating or multi value attributes within a single column.

Second Normal Form (2NF) - The second normal form is satisfied too. It ensures all attributes are fully functionally dependent on the primary key. It removes partial dependencies. In the raw flat dataset, this rule is violated because the given composite primary key is composed (**NU_ANO_CENSO** and **CO_MUNICIPIO**). However, there are attributes like **NO_UF** and **SG_UF** that actually have functional dependency with the municipality code (**CO_MUNICIPIO**) only. These attributes are then pulled out into a separate **LOCATION** entity with its own primary as in Figure 2.3 shown. The 6 new entities are now in 2NF which means that all non-prime attributes are fully dependent on their primary key.

Third Normal Form (3NF) - A schema is in third normal form if it is now in 2NF and there are no transitive dependencies. Transitive dependencies explained that there should not be any non-prime attribute that is dependent on another non-prime attribute. However, after designing the phase 2 data model, there is still a transitive dependency between municipality code (**CO_MUNICIPIO**) and state code (**CO_UF**) which then determines state name (**NO_UF**) and the state abbreviation (**SG_UF**). Since it is still not the best data model, the star schema taught by the tutorial should be the best data model.

3.0 Data Preparation

3.1 Prepare environment and data for analysis

This data preparation phase explains the pre-processing steps to load, clean and restructure the 2.2GB of raw unnormalized data using Apache Spark and Python. This phase mainly is to address major data engineering anomalies such as string encoding mismatches, handling missing values and manual type casting. Before this process, we need to make sure the spark runs in a correct and suitable environment and libraries.

3.1.1 Environment Setup and Library Initialization

```
# Clear ALL spark-related environment variables
for key in ['SPARK_HOME', 'PYSPARK_SUBMIT_ARGS']:
    if key in os.environ:
        del os.environ[key]

# Set correct ones
os.environ['HADOOP_HOME'] = 'C:\\hadoop'
os.environ['PYSPARK_PYTHON'] = 'C:\\Python311\\python.exe'
os.environ['PYSPARK_DRIVER_PYTHON'] = 'C:\\Python311\\python.exe'

print("✅ Environment ready!")
```

Figure 3.1 Setting up environment and library initialization

During the tutorial session, there will be conflicts on Windows that cause crashes since the laptop has multiple installed versions of Spark and Python. That is why the system environment must be correctly configured before any Spark operation begins to avoid the crashes. Figure 3.1 shows the codes to remove all conflicting **SPARK_HOME** variables and sets **HADOOP_HOME** to point to the WinUtils installation. It is a utility required by Spark to perform file system operations on Windows. Next, the **PYSPARK_PYTHON** variables are used to make sure that both the driver and executor processes use the correct **PYTHON 3.11** installation rather than the default Python 3.14 which is actually incompatible with PySpark 3.5.1.

3.1.2 Library Imports

```
from pyspark.sql import SparkSession
import pyspark.sql.functions as F

import psycopg2
from datetime import datetime

from itertools import chain
```

Figure 3.2 Importing libraries used

Based on the Figure 3.2, this cell imports all the necessary libraries before any processing begins. The SparkSession is an access point for all the functionality to use Apache Spark. Next, psycopg2 is a Python library for connecting to and executing SQL commands in PostgreSQL. It will be used later to set primary keys. Lastly, the chain from itertools is used later to flatten lists when constructing the fact table column list.

3.1.3 Creating the SparkSession

```
spark = SparkSession.builder\
        .appName("CensoEscolarStarSchema")\
        .config("spark.sql.shuffle.partitions", "4")\
        .config("spark.driver.memory", "8g")\
        .config("spark.executor.memory", "8g")\
        .getOrCreate()
```

Figure 3.3 Spark operation to create spark session

Based on the Figure 3.3, the SparkSession is an access point for all the functionality to use Apache Spark. Each configuration is designed for a specific purpose. The **.appName("CensoEscolarStarSchema")** line is to name the Spark application in the Spark UI. So, the Spark name for this tutorial is CensoEscolarStarSchema. Next, the **.config("spark.sql.shuffle.partitions", "4")** is used to reduce the number of CPU cores available to avoid over partitioning on our local machine. The **.config("spark.driver.memory", "8g")** and **.config("spark.executor.memory", "8g")** lines allocate 8GB of RAM for the Spark driver to process plus execute the dataset of approximately 2.2GB. Lastly, the **.getOrCreate()** launches Spark in local mode.

3.1.4 Extracting Data

```python
import os
import zipfile
import urllib.request
import ssl
import glob
import shutil

# Target directory
TARGET_DIR = r"C:\Users\HUAWEI\ApacheSparkWork\posts\etl_spark_scholar_census\data"

def main():
    os.makedirs(TARGET_DIR, exist_ok=True)

    # Loops through years 2010 to 2021
    for year_num in range(2010, 2022):
        year = str(year_num)

        print("\n" + "="*50)
        print(f"PROCESSING CENSUS DATA FOR YEAR: {year}")
        print("="*50)

        # Dynamically changes the URL and file names based on the current year in the loop
        url = f"https://download.inep.gov.br/dados_abertos/microdados_censo_escolar_{year}.zip"
        zip_path = os.path.join(TARGET_DIR, f"{year}.zip")
        extract_path = os.path.join(TARGET_DIR, year)
        output_csv = os.path.join(TARGET_DIR, f"{year}.csv")

        print(f"Downloading {year} data (ignoring SSL verification)...")
        try:
            # Bypasses local SSL certificate validation errors common with government servers
            context = ssl._create_unverified_context()

            # Mimics a real web browser header to prevent the server from blocking the automated download
            req = urllib.request.Request(
                url,
                headers={'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64)'}
            )

            # Opens the URL connection and streams the binary content directly into a local zip file
            with urllib.request.urlopen(req, context=context) as response, open(zip_path, 'wb') as out_file:
                shutil.copyfileobj(response, out_file)

            print(f"Download complete for {year}.")
        except Exception as e:
            print(f"❌ Download failed for {year}: {e}")
            continue  # Crucial: Skips the rest of this year's steps and moves to the next year if a download fails


        print(f"Extracting {year} ZIP file...")
        try:
            with zipfile.ZipFile(zip_path, 'r') as zip_ref:
                zip_ref.extractall(extract_path)
        except Exception as e:
            print(f"❌ Extraction failed for {year}: {e}")
            if os.path.exists(zip_path):
                os.remove(zip_path)
            continue

        # Uses glob to scan all subdirectories recursively because internal folder structures change across years
        search_pattern = os.path.join(extract_path, "**", "*.CSV")
        # Combines uppercase (.CSV) and lowercase (.csv) results to ensure case-insensitive safety on Windows
        csv_files = glob.glob(search_pattern, recursive=True) + \
                    glob.glob(search_pattern.replace("*.CSV", "*.csv"), recursive=True)

        if csv_files:
            print(f"Found CSV file path target: {csv_files[0]}")
            # Pulls the nested CSV out of the messy extracted folders, moves it to the main directory, and renames it
            shutil.move(csv_files[0], output_csv)
            print(f"✅ Successfully created target file: {output_csv}")
        else:
            print(f"❌ CSV file not found inside the {year} folder structure.")

        print(f"Cleaning up temporary compression structures for {year}...")
        # Deletes the original .zip and the extracted folder to save disk space since we already saved the clean CSV
        if os.path.exists(zip_path):
            os.remove(zip_path)
        if os.path.exists(extract_path):
            shutil.rmtree(extract_path)
        print(f"Cleanup sequence finalized for {year}.")

    print("\n All operations completed across the 2010-2021 timeline.")

if __name__ == "__main__":
    main()
```

Figure 3.4 Updated code in extracting data

Figure 3.4 shows the corrected code in extracting data. The tutorial's original code relies on os.system() to execute Linux or Bash terminal commands. However, on windows these commands like **wget** and **mv** could not be used. That is why there is a huge difference in the code between the tutorial's and the current one. To make the extraction process robust and cross-platform, Python standard libraries are used to download, unzip, move and clean up logic.

3.1.5 Directory and Files Checking

```
#additional checking
import os
print(os.getcwd())
```

```
C:\posts-main\etl_spark_scholar_census
```

Figure 3.6 Working Directory Check

Figure 3.6 shows the code as a verification step to confirm that the notebook is running from the correct working directory (C:\posts-main\etl_spark_scholar_census).

```
#additional checking
import os
files = os.listdir("./data")
print(files)
```

```
['2010.csv', '2011.csv', '2012.csv', '2013.csv', '2014.csv', '2015.csv', '2016.csv', '2017.csv', '2018.csv', '20
19.csv', '2020.csv', '2021.csv', 'censo_escolar.parquet']
```

Figure 3.7 Verify CSV Files Exist

Figure 3.7 shows another verification step that is used to double check all files in the ./data folder to confirm that all CSV files were downloaded correctly. The output confirms 12 CSV files (2010–2021) are present.

3.2 Transforming CSV to Parquet

3.2.1 Read CSV Files

```
import glob

csv_files = glob.glob("C:/posts-main/etl_spark_scholar_census/data/*.csv")

data_csv = (
    spark
    .read
    .format("csv")
    .option("header", "true")
    .option("inferSchema", "false")
    .option("delimiter", ";")
    .option("encoding", "iso-8859-1")
    .load(csv_files)
)
```

Figure 3.8 Reading CSV Files Using glob

In Figure 3.8, glob libraries are imported to resolve Windows-specific wildcard limitations. It reads all CSV files into a single Spark DataFrame before passing it to Spark. The **.option("header","true")** is used to always let the first row as column names, **inferSchema=false** to let all columns load as strings initially for better performance. Next, **delimiter=";"** because Brazilian government data uses semicolons rather than commas and **encoding="iso-8859-1"** to handle Portuguese special characters.

3.2.2 Write to Parquet

```
data_csv.write.parquet("./data/censo_escolar.parquet")
```

Figure 3.9 Write the data into parquet

Figure 3.9 shows the line that triggers the actual data processing part. Spark reads all CSV files and converts them into a single new Parquet file. Parquet is a columnar binary storage format that has about 5 times compression of files compared to raw CSV. It provides faster read performance that really saves time.

3.2.3 Read from Parquet

```
data = (
    spark
    .read
    .format("parquet")
    .load("./data/censo_escolar.parquet/")
)
```

Figure 3.10 Read data from parquet

From Figure 3.10, the Parquet file is loaded back into a fresh Spark DataFrame named data after conversion into parquet file. Now, this "data" variable becomes the primary source for all remaining transformation steps.

3.2.4 Verify Parquet Data

```
#additional checking
print(data.count())
print(len(data.columns))
```

```
2792984
370
```

Figure 3.11 Check the rows and columns of data

Figure 3.11 shows the code that is used to know the total rows and columns that have been converted. There are 2792984 records and 370 columns were preserved.

4.0 Modeling

4.1 Star Schema Approach

This project implements a multidimensional data model, star schema, that is frequently used in data warehousing and business intelligence projects. The star schema was selected over a normalized relational model since it supports the analytical queries such as aggregating millions of records by year, location or educational facility rather than transactional operations. In addition, this model improves the query efficiency for certain types of aggregations compared to a fully normalized relational model which requires multiple levels of joins as star schema keeps all dimension information only one join away from the facts table.

The star schema design consists of 13 tables which includes 1 central fact table, FACR_CENSO_ESCOLAR and 12 surrounding dimension tables that cover location, school

type, and facility information. The visualization of the architecture is shown in Figure 4.1 below.

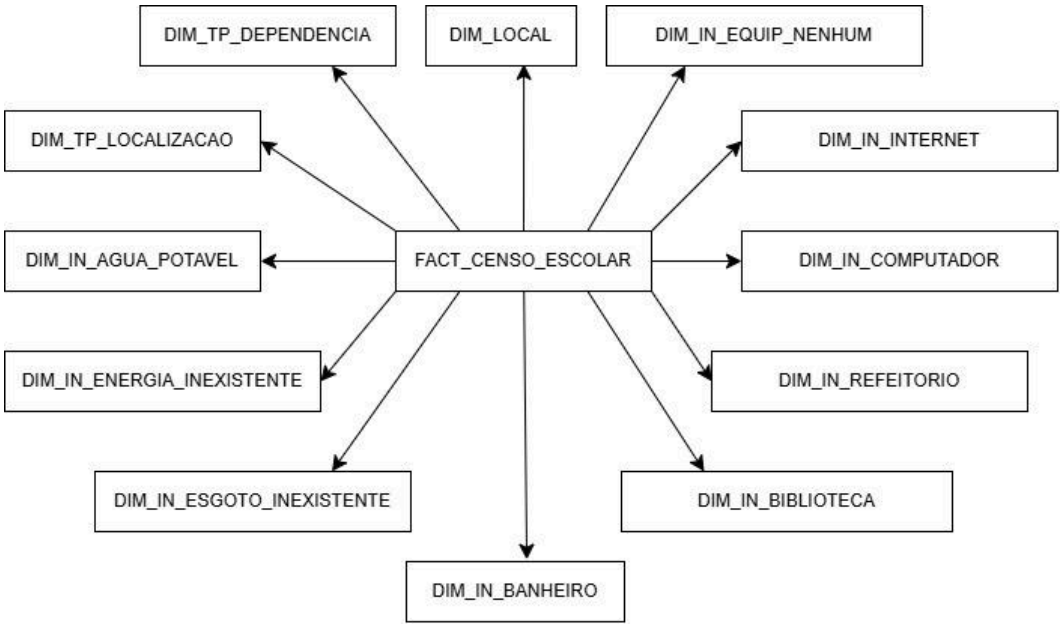

Figure 4.1 Star Schema Architecture Design

4.2 Hierarchies in Star Schema

4.2.1 Geographic hierarchy

Geographic hierarchy exists within the dim_local dimension table as the location data can be structured in levels from large to small. It contains five columns that naturally represent 2 geographic levels within a single table as shown in Table 4.1 below.

Column	Description	Level	Example
NO_UF	Full name of state	State	São Paulo
SG_UF	Abbreviation of state	State	SP
CO_UF	Numeric code of state	State	35
NO_MUNICIPIO	Full name of city	City	Guarulhos
CO_MUNICIPIO	Numeric code of city	City	3518800

Table 4.1 Overview of dim_local Dimension

This means data can analyzed at different geographic levels using the same single join between the fact table and dim_local dimension table:

Brazil (Country) → State (NO_UF / SG_UF)→ City (NO_MUNICIPIO / CO_MUNICIPIO) → School record in fact table

4.2.2 Time Hierarchy

Time Hierarchy exists in the nu_ano_censo column in the fact table that represents the census year from 2010 to 2021. Although it stored in a single column, it enables trend analysis over time at different level:

All Years (2010–2021) → Individual Year (2010, 2011, 2012 ... 2021)

4.3 Connection between PostgreSQL Database Server and Spark Session

To connect the Spark with the PostgreSQL database, two different types of connection were created to manage the large scale of data transfer while allowing for specific updates to the database structure.

```
POSTGRES_USER = "censo"
POSTGRES_PASSWORD = "123"
POSTGRES_DB = "censo_escolar"

# Used to connect to the PostgreSQL database server
# in spark session
POSTGRES_CONFIG = {
    "url":f"jdbc:postgresql://localhost:5432/{POSTGRES_DB}",
    "properties":{
        "user":POSTGRES_USER,
        "password":POSTGRES_PASSWORD,
        "driver":"org.postgresql.Driver",

    },
}
```

Figure 4.2 Setting up Spark for Data Transfer

Figure 4.2 shows the connection details and address router were set to specific defined terms and location. JDBC was used to set up Spark to move large data to the database quickly.

```
conn = psycopg2.connect(
    host="localhost",
    port="5432",

    dbname=POSTGRES_DB,
    user=POSTGRES_USER,
    password=POSTGRES_PASSWORD
)
```

Figure 4.3 Setting up Direct Link for Database Changes

Figure 4.3 shows another direct connection was created with psycopg2 to manage and ensure the data stays organized and accurate. It allows users to use the same login details to open the direct path to the database every time.

4.4 Dimension Implementation

4.4.1 Dimension Table Structure

These dimension tables serve as the star schema's foundation which contains the categories and attributes for each entity in the study. Table 4.2 below shows the 12 dimension tables with their respective row volumes and descriptions that will be produced.

Dimension Table	Description	Rows
dim_local	All unique state and city combinations	~5570
dim_tp_dependencia	Federal, State, Municipal, Private	4
dim_tp_localizacao	Urban, Rural	2
dim_in_banheiro	Has bathroom: Yes/No	2
dim_in_internet	Has internet: Yes/No	2
dim_in_agua_potavel	Has water Yes/No	2
dim_in_biblioteca	Has library: Yes/No	2
dim_in_refeitorio	Has cafeteria: Yes/No	2
dim_in_computador	Has computers: Yes/No	2
dim_in_energia_inexistente	No electricity: Yes/No	2
dim_in_esgoto_inexistente	No sewage: Yes/No	2
dim_in_equip_nenhum	No equipment: Yes/No	2

Table 4.2 Dimension Table Structure

The dim_local table contains approximately 5570 rows as it stores unique combinations of state and city across Brazil with 26 states and thousands of municipals, the number of unique location combinations is high. In contrast, each facility dimension table contains only 2 rows that represent binary values 0(Yes) and 1(No) to determine whether a school has that facility or not. The dim_tp_dependencia includes 4 rows representing 4 types of school and the dim_tp_localizacao includes 2 rows that represent Urban and Rural classification. These low cardinality dimensions allow for fast joins between fact tables and dimension tables during analytics.

The final PostgreSQL database implementation should completely match this structure to ensure all details are correctly recorded before loading any data.

4.4.2 Dimension Table Configuration

A clear set of rules is crucial for every table before any data is moved since all information should stay organized and accurate to avoid any mismatches between the raw data and the final star schema. Hence, a master configuration dictionary named DIMENSION_TABLES_CONFIG was used to define the 12 dimension tables. Each entry includes the name of the table, columns of dimension, and the data types of those columns as shown in the Figure 4.4 below.

```
INTEGER_DIMENSIONS = [
    "TP_DEPENDENCIA",
    "TP_LOCALIZACAO",
    "IN_AGUA_POTAVEL",
    "IN_ENERGIA_INEXISTENTE",
    "IN_ESGOTO_INEXISTENTE",
    "IN_BANHEIRO",
    "IN_BIBLIOTECA",
    "IN_REFEITORIO",
    "IN_COMPUTADOR",
    "IN_INTERNET",
    "IN_EQUIP_NENHUM"
]

DIMENSION_TABLES_CONFIG = {
    "DIM_LOCAL":{
        "fields": [
            {"field":"NO_UF", "type":"string",},
            {"field":"SG_UF", "type":"string",},
            {"field":"CO_UF", "type":"string",},
            {"field":"NO_MUNICIPIO", "type":"string",},
            {"field":"CO_MUNICIPIO", "type":"string",}
        ]
    },
}

DIMENSION_TABLES_CONFIG.update(
    {
        "DIM_"+dimension.upper():{
            "fields": [
                {"field":dimension, "type":"integer"}
            ]
        }
        for dimension in INTEGER_DIMENSIONS
    }
)
```

Figure 4.4 Configuration Dictionary for Dimension Table

This approach allows all dimension tables built by using a single for loop,which makes the code more reusable rather than rewrite separate code for each table.

4.4.3 Build Dimension Table by Spark

After connecting to the PostgreSQL database, the provided code as shown in Figure 4.5 below creates the dimensions. Spark will create a table with the name of the dimension and the columns by using configuration in DIMENSION_TABLES_CONFIG.

```
for table_name, table_config in DIMENSION_TABLES_CONFIG.items():
    print(f"[{datetime.now()}] Writing {table_name}")
    data\
    .select([F.col(field["field"]).cast(field["type"]).alias(field["field"])
            for field in table_config["fields"]])\
    .distinct()\
    .withColumn("id", F.monotonically_increasing_id())\
    .write\
    .jdbc(**POSTGRES_CONFIG,table=table_name,mode="overwrite")
```

Figure 4.5 Spark Operation to Build Dimension Table

Table 4.3 summaries the five key Spark operations used in this process with their respective functions and purposes. Each operation plays a specific role in transforming the raw data into a clean and structured dimension table.

Spark Operation	Function	Purpose
.select()	Select relevant columns for dimension	Reduces data sizes
.cast()	Convert string columns to accurate types(integers)	Needed since inferSchema was false at first
.distinct()	Keeps only unique rows	Dimensions should have no duplicates
.withColumn("id", monotonically_increasing_id())	Assigns unique surrogate key	Primary key for each dimension
.write.jdbc(mode="overwrite")	Writes to PostgreSQL and creates table automatically	Spark auto_creates table structure

Table 4.3 Explanation of Spark Operation

When writing was completed, the primary key was added via psycopg2 functions by executing the ALTER TABLE command as shown in Figure 4.6 below.

```
cursor = conn.cursor()
cursor.execute(
    f"ALTER TABLE {table_name} ADD PRIMARY KEY (id);"
)
cursor.close()
conn.commit()
```

Figure 4.6 Add Primary Key Function

4.4.4 Dimension Table Results

The dimension table was successfully created and the result can be shown in 3 platforms listed below.

1) Python Execution Log

```
[2026-05-14 18:22:22.401288] Writing DIM_LOCAL
[2026-05-14 18:22:26.905043] Wrote DIM_LOCAL
[2026-05-14 18:22:26.943968] Added primary key to DIM_LOCAL
[2026-05-14 18:22:26.944084] Done
[2026-05-14 18:22:26.944102] Writing DIM_TP_DEPENDENCIA
[2026-05-14 18:22:28.406476] Wrote DIM_TP_DEPENDENCIA
[2026-05-14 18:22:28.440522] Added primary key to DIM_TP_DEPENDENCIA
[2026-05-14 18:22:28.440678] Done
```

Figure 4.7 Python Execution Log

The execution log produced as shown in Figure 4.7 above, it showed 2 dimension tables had completed successfully while the other 10 dimension tables had the same successful result at the end.

2) Spark UI (localhost:4040)

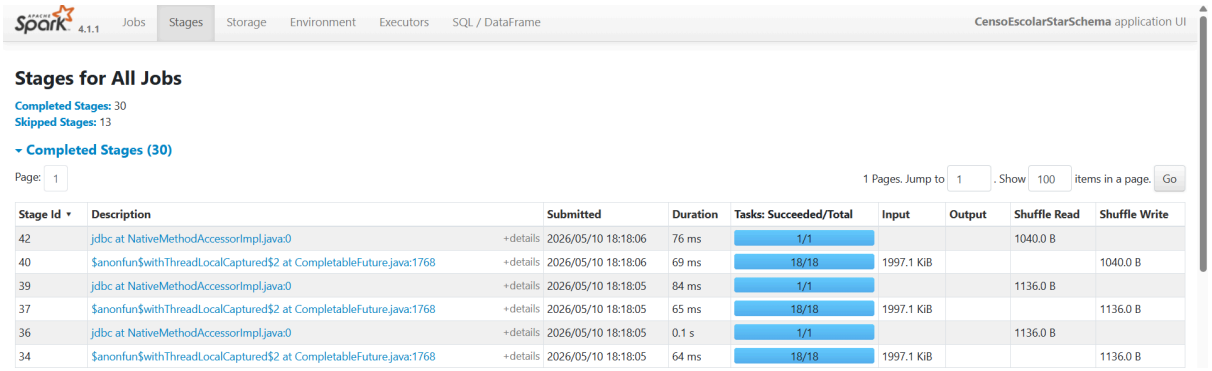

Figure 4.8 30 Completed Stages in Spark UI

The Spark UI completed stages updates to 30 completed jobs after dimension tables created successfully as shown in Figure 4.8 above. Each dimension ran approximately 2 to 3 jobs that included reading, distinct operation and writing to PostgreSQL via JDBC.

3) Adminer (localhost:8080)

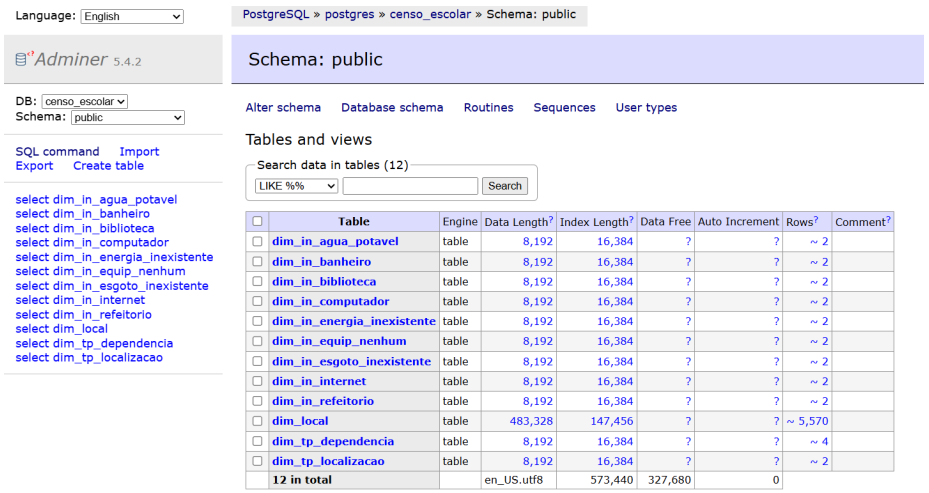

Figure 4.9 New Dimension Table in Adminer

After the cell was complete, the Adminer showed 12 new tables created inside the censo_escolar database as shown in Figure 4.9 above. It shows the information of each dimension such as datalength, index length and rows.

4.5 Fact Table Implementation

4.5.1 Fact Table Structure

The fact table FACT_CENSO_ESCOLAR is the central table of the star schema, which includes quantitative measures and foreign keys referencing each dimension table. Table 4.4 shows the 15 metric columns of measures with their respective description will be created and Table 4.5 represents the 12 foreign keys with their respective dimension table references will be added inside the fact table structure.

Column	Description	Type
QT_DOC_BAS	Number of teachers in Basic Education	INTEGER
QT_DOC_INF	Number of teachers in Kindergarden	INTEGER
QT_DOC_FUND	Number of teachers in Primary School	INTEGER
QT_DOC_MED	Number of teachers in High School	INTEGER
QT_MAT_BAS	Total student enrollments (Basic Education)	INTEGER
QT_MAT_INF	Enrollment in Kindergarden	INTEGER
QT_MAT_FUND	Enrollment in Primary School	INTEGER
QT_MAT_MED	Enrollment in High School	INTEGER
QT_MAT_BAS_ND	Enrollment - Race Not Declared	INTEGER
QT_MAT_BAS_BRANCA	Enrollment - White Race	INTEGER
QT_MAT_BAS_PRETA	Enrollment - Black Race	INTEGER
QT_MAT_BAS_PARDA	Enrollment - Mixed Race	INTEGER
QT_MAT_BAS_AMARELA	Enrollment - Asian Race	INTEGER
QT_MAT_BAS_INDIGENA	Enrollment - Indigenous	INTEGER
NU_ANO_CENSO	Census Year	INTEGER

Table 4.4 Measures in Fact Table FACT_CENSO_ESCOLAR

The datatype of measure columns are designed into INTEGER type since the values they store are year, enrollment and teachers count with the whole number. For example, the total enrollment count QT_MAT_BAS in one year might be approximately 2.1 billion which does not exceed the maximum value that INTEGER can hold. This proved that INTEGER is a suitable choice for these columns.

Foreign Key Column	References	Type
ID_DIM_LOCAL	dim_local	BIG INTEGER
ID_DIM_TP_DEPENDENCIA	dim_tp_dependencia	BIG INTEGER
ID_DIM_TP_LOCALIZACAO	dim_tp_localizacao	BIG INTEGER
ID_DIM_IN_AGUA_POTAVEL	dim_in_agua_potavel	BIG INTEGER
ID_DIM_IN_BANHEIRO	dim_in_banheiro	BIG INTEGER
ID_DIM_IN_EQUIP_NENHUM	dim_in_equip_nenhum	BIG INTEGER
ID_DIM_IN_BIBLIOTECA	dim_in_biblioteca	BIG INTEGER
ID_DIM_IN_REFEITORIO	dim_in_refeitorio	BIG INTEGER
ID_DIM_IN_COMPUTADOR	dim_in_computador	BIG INTEGER
ID_DIM_IN_INTERNET	dim_in_internet	BIG INTEGER
ID_DIM_IN_ENERGIA_INEXISTENTE	dim_in_energia_inexistente	BIG INTEGER
ID_DIM_IN_ESGOTO_INEXISTENTE	dim_in_esgoto_inexistente	BIG INTEGER

Table 4.5 Foreign Keys in Fact table FACT_CENSO_ESCOLAR

However, the datatype of foreign key columns are designed into BIGINT type as the surrogate keys mapped to the dimension table rows were created with Spark monotonically_increasing_id() function. It produces large 64-bit integer values that potentially exceed the range of INTEGER type. Hence, BIGINT is used to ensure that the fact table's foreign key references can store and match these large surrogate key values without any errors.

This empty structure serves as the basic blueprint for data model and will stay unpopulated until the mapping rules and final data transfer processes are completed.

4.5.2 Fact Table Configuration

Similarly, a clear design for the fact table is required to ensure all numerical data is appropriately linked to descriptive categories.

```
FACT_TABLE_NAME = "FACT_CENSO_ESCOLAR"

FACT_COLUMNS = [
    "QT_DOC_BAS",   # Número de Docentes da Educação Básica
    "QT_DOC_INF",   # Número de Docentes da Educação Infantil
    "QT_DOC_FUND",  # Número de Docentes do Ensino Fundamental
    "QT_DOC_MED",   # Número de Docentes do Ensino Médio

    "QT_MAT_BAS",   # Número de Matrículas na Educação Básica (TOTAL)
    "QT_MAT_INF",   # Número de Matrículas na Educação Infantil
    "QT_MAT_FUND",  # Número de Matrículas no Ensino Fundamental
    "QT_MAT_MED",   # Número de Matrículas no Ensino Médio

    "QT_MAT_BAS_ND",        # Número de Matrículas na Educação Básica - Cor/Raça Não Declarada
    "QT_MAT_BAS_BRANCA",    # Número de Matrículas na Educação Básica - Cor/Raça Branca
    "QT_MAT_BAS_PRETA",     # Número de Matrículas na Educação Básica - Cor/Raça Preta
    "QT_MAT_BAS_PARDA",     # Número de Matrículas na Educação Básica - Cor/Raça Parda
    "QT_MAT_BAS_AMARELA",   # Número de Matrículas na Educação Básica - Cor/Raça Amarela
    "QT_MAT_BAS_INDIGENA",  # Número de Matrículas na Educação Básica - Cor/Raça Indígena

    "NU_ANO_CENSO"
]

FACT_CONFIG = {
    fact:{
        "fields": [
            {"field":fact, "type":"integer"}
        ]
    }
    for fact in FACT_COLUMNS
}
```

Figure 4.10 Configuration Dictionary for Fact Table

A configuration set including FACT_COLUMNS, FACT_CONFIG is created to define the quantitative metrics of the study as shown in Figure 4.10 above.

```
DIMENSION_ID_CONFIG = {
    table_name:[
        field['field']
        for field
        in table_fields['fields']
    ]
    for table_name, table_fields in DIMENSION_TABLES_CONFIG.items()
}

FACT_TABLE_ALL_COLUMNS_ORDERED = FACT_COLUMNS + list(map(lambda col:"ID_"+col, DIMENSION_ID_CONFIG.keys()))
```

Figure 4.11 Continuous Configuration Dictionary for Fact Table

Based on Figure 4.11, the DIMENSION_ID_CONFIG is used to identify the specific columns to ensure the fact table can be linked to the 12 dimensions properly that had been created just before. The system can automatically arrange the data later in appropriate order by organising the details into FACT_TABLE_COLUMNS_ORDERED. This function creates the final column list that combines the metrics with the foreign key ID.

4.5.3 Build Fact Table by SQL Script

```
comma_break_line = ",\n\t\t\t"
facts_table_sql = f"""
    CREATE TABLE IF NOT EXISTS {FACT_TABLE_NAME} (
        id SERIAL PRIMARY KEY,
        {
            comma_break_line.join(
                [
                    f"{field} INTEGER"
                    for field in FACT_COLUMNS
                ]
                +[
                    f"ID_{dim_table} BIGINT"
                    for dim_table in DIMENSION_ID_CONFIG.keys()
                ]
            )
        }
    );

    -- Adding Foreign Keys
    ALTER TABLE {FACT_TABLE_NAME}
    {
        comma_break_line.join(
            [
                f"ADD CONSTRAINT {FACT_TABLE_NAME}_{dim_table}_fk FOREIGN KEY (ID_{dim_table}) REFERENCES {dim_table}(id)"
                for dim_table in DIMENSION_ID_CONFIG.keys()
            ]
        )
    }
"""
```

Figure 4.12 SQL Script for Fact Table Structure

Since the facts table has complex relationships, the table structure is manually created by using a custom SQL script to ensure every detail is precisely defined as shown in Figure 4.12 above. The facts_table_sql script creates the empty table to set up the metrics as integers and the dimension links as large integers. Additionally, foreign key constraints were implemented to prevent any record in the fact table from referencing a dimension ID that does not exist.

```
print(f"[{datetime.now()}] Creating facts table")

cursor = conn.cursor()
try:
    cursor.execute(facts_table_sql)
    cursor.close()
    conn.commit()
except Exception as e:
    print(e)
    conn.rollback()
    cursor.close()
else:
    print(f"[{datetime.now()}] Created facts table")
    print(f"[{datetime.now()}] Done")
```

Figure 4.13 PostgreSQL Table Creation Process

Once the structure is defined, the command is sent to the PostgreSQL server to finalize the structure of the fact table. As shown in Figure 4.13, the database cursor is used to execute the facts_table_sql scripts to build the table and implement the necessary rules inside it with error handling and status updates to ensure the process is completed without any failures.

4.5.4 Fact Table Result

After the execution of the fact table creation script, the results were verified using the Adminer to ensure that the star schema was successfully configured.

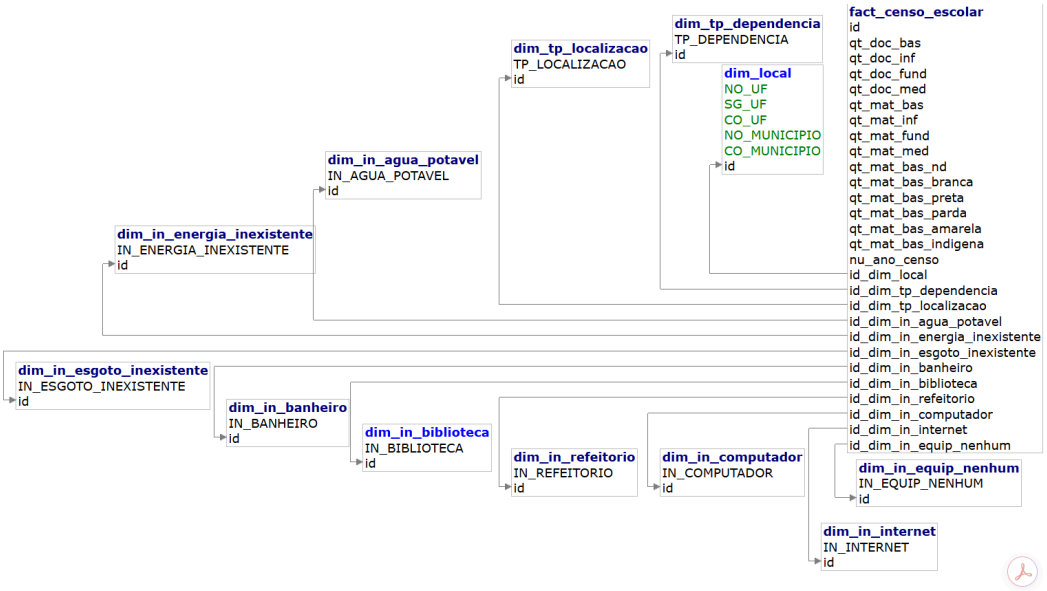

Figure 4.14 Star Schema Relationship Diagram

As shown in Figure 4.14, the fact_censo_escolar table is at the center of the architecture, which acts as the primary table that connects to all 12 dimension tables. The internal contents of the fact table includes all quantitative metrics and connection keys are correctly displayed. This centralized layout ensures that each record is properly linked to its corresponding details which have provided a complete and well-organized view of the entire dataset. The final multidimensional star schema is confirmed to be ready and fully operational.

4.6 Populating Fact Table

4.6.1 Population Process

After completing the star schema's structural configuration and mapping rules, the database is still completely empty with no data records within the central hub. Hence, the next critical step is to populate the fact table to bring the architecture to life. Populating a table is the process of filling an empty database structure with real records and within this stage, millions of rows of processed data are transferred from the temporary Spark environment to the PostgreSQL database.

```
facts_data = data\.select([*chain(*DIMENSION_ID_CONFIG.values(), FACT_CONFIG.keys())]

for table_name, table_fields in DIMENSION_ID_CONFIG.items():
    # Read the dimension data from Postgres
    dim_table = spark.read\.jdbc(**POSTGRES_CONFIG,table=table_name,)\
        .withColumnRenamed("id", f"ID_{table_name}")

    # Join the dimension data with the fact data
    facts_data = facts_data\
        .join(dim_table,on=table_fields,how="left")\
        .drop(*table_fields)
```

Figure 4.15 Populating Code

The fact table was populated through a sequence of LEFT JOINs through Spark to replace raw attributes values with their corresponding dimension surrogate keys as shown in the Figure 4.15 code section. For each of the 12 dimension tables, Spark reads the dimension table back from PostgreSQL, then joins it with the fact data. At this point, Spark did not process any actual data yet, it only recorded the transformations as a logical plan, the actual computation was only executed when the .write.jdbc() action was called. This approach is more efficient than performing each transformation step directly as Spark can optimize the full sequence of 12 joins before any data moves, it minimises unnecessary data processing and memory usage.

```
facts_data\
    .select(
        [F.col(c).cast("integer") if c in FACT_COLUMNS else F.col(c)
         for c in FACT_TABLE_ALL_COLUMNS_ORDERED]
    )\
    .write\
    .jdbc(
        **POSTGRES_CONFIG,
        table=FACT_TABLE_NAME,
        mode="append"
    )
```

Figure 4.16 Continuous Populating Code

Before the transfer began, the data columns were manually converted to integers since inferSchema:false in data preparation phase had saved all the values as strings in Parquet as shown in Figure 4.16. Without the casting, PostgreSQL will refuse the insert as the fact table columns are declared as INTEGER types. The .write.jdbc() action executed and Spark started to write all 2.79 million rows of transformed fact data into PostgreSQL database via JDBC in append mode.

4.6.2 Populating Result

1) Spark UI (localhost:4040)

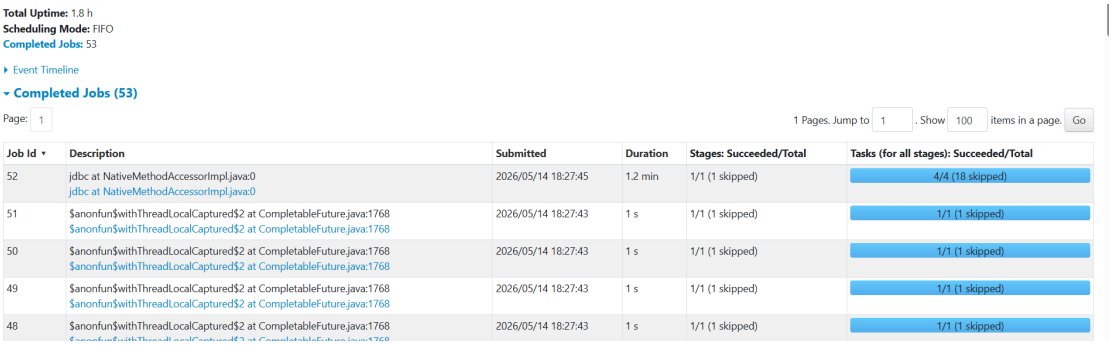

Figure 4.17 53 Completed Stages in Spark UI

The Spark UI completed stages updates to 53 completed jobs from previously 30 completed jobs after the entire modelling pipeline completed as shown in Figure 4.17 above. The populating took approximately 1.2 minutes to complete which made it the longest single job in the pipeline and it represents the size of writing 2.79 million rows to PostgreSQL via JDBC.

2) Adminer (localhost:8080)

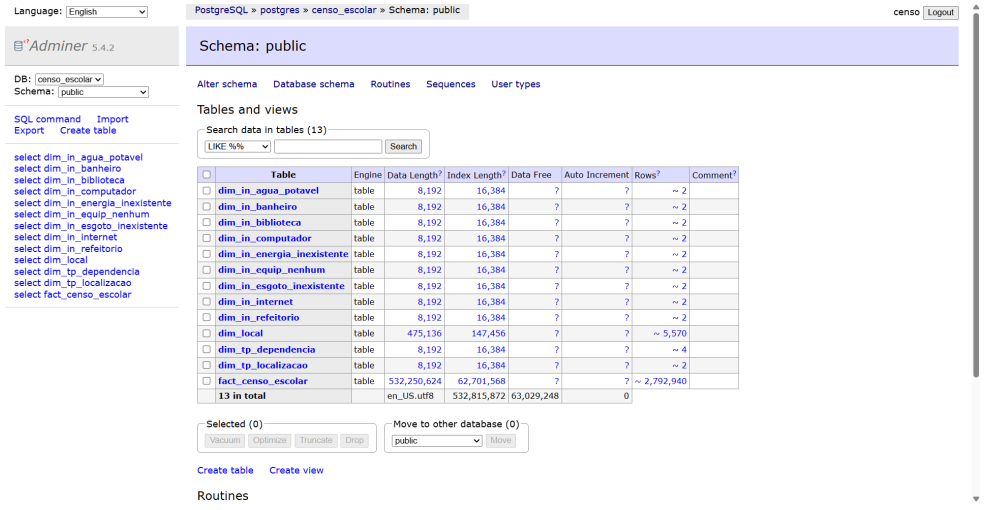

Figure 4.18 Final Database Table List in Adminer

In Figure 4.18, the database schema now contains a total of 13 tables, representing an increase from the 12 dimension tables previously created. The fact_censo_escolar table serves as the central fact table and it holds approximately 2,793,022 rows of data, making it the largest dataset in the system. This confirms that the transformation from raw data into a multidimensional star schema is successfully completed.

5.0 Evaluation

5.1 Pipeline Evaluation

The ETL process was implemented, and the Brazilian school census data for 12 years (2010 to 2021) was successfully processed. It reads 2,792,984 lines from a CSV file, and saves them in Parquet format, creates a star schema of 12 dimensional tables and a fact table, and loads the data to the PostgreSQL database. All stages were completed smoothly without serious errors, which proves that Apache Spark can efficiently process data of this scale on local machines.

5.2 Dashboard Evaluation

Refer to Figure 5.1, a Metabase dashboard called "Censo Escolar Dashboard" is created to evaluate if the process has attained the business goals established earlier. The main visuals on this dashboard are a line graph of the total number of enrollments by each year, a summary card of the research situation for the year 2021, a total enrollment statistic for 2021, a map of the total enrollment numbers by state, and a table of the top 10 cities for total enrollment in 2021.

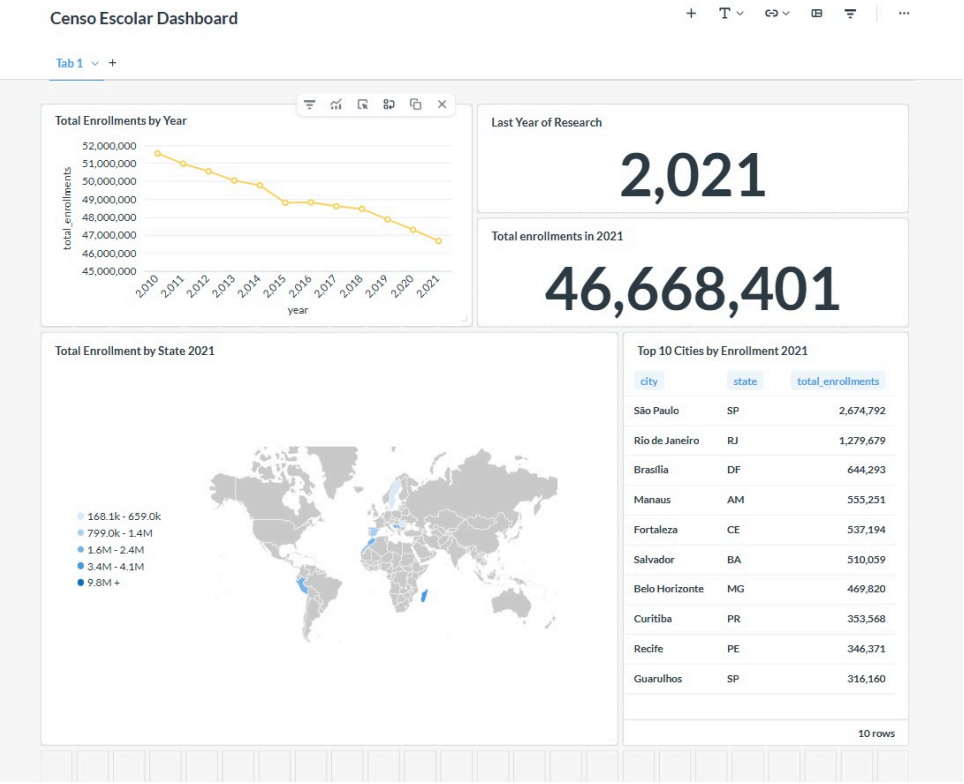

Figure 5.1: Censo Escolar Dashboard in Metabase

The dashboard reveals several important pieces of information from the data. The overall downward trend is seen in the total number of enrolments, which decreased from 51,549,889 in 2010 to 46,668,401 in 2021, a decline of about 4.9 million students. This trend may reflect the population changes in Brazil during the same period, such as the decline in the birth rate.

Regarding geographical distribution, 2,674,792 people registered in 2021 in the state of São Paulo, 1,279,679 people in the state of Rio de Janeiro and 644,293 people in the Federal District – Brasília. The remaining ten cities (Manaus, Fortaleza, Salvador, Belo Horizonte, Curitiba, Recife and Gurupi) report between 316,160 and 555,251 students, demonstrating the focus of students in the major cities of Brazil.

5.3 Limitations

The process was successfully implemented, but a few limitations were identified. It runs in batch mode and must be manually re-run every year when new data becomes available. Moreover, there might be empty nulls for some of the numerical columns in the fact table that will influence the aggregated indicators. The disruption to the data collection process due to the COVID-19 pandemic may have influenced the data from 2021, further limiting the comparability of findings derived from this year swith any previous year. Finally, this process was built and tested on a Windows system, which requires additional configuration steps such as WinUtils and Hadoop environment variables. On the Linux system, these steps are not necessary.

5.4 Conclusion

Overall, the entire ETL pipeline achieved all business objectives that were set at the beginning of the project. By using Apache Spark as the main data processing engine, 2,792,984 rows of original Brazilian school census data covering 12 years were efficiently extracted, transformed, and loaded without any major failures during the process.

Turning the original CSV into Parquet set the team up well for optimization as parquet files are about 5 times smaller than the original CSV. Converting files to parquet also provided faster read performance at the time of transformation.The Star Schema design consists of 12 dimension tables and one Fact table containing 15 metrics and 12 foreign key references, ensuring optimized query performance. Using the star schema design, Metabase was able to run aggregating queries very fast on 7 million+ rows.

Metabase was able to answer all 4 business objectives directly, which proves the pipeline worked correctly end-to-end. In short, the ETL pipeline design project showed how one can use Apache Spark, PostgreSQL, and Metabase to convert raw government data into clear business intelligence insights into Brazilian basic education between 2010 and 2021.

Reflection

Chau Ying Jia

This tutorial taught us to make a complete ETL pipeline using platforms like docker, Apache Spark, PySpark, PostgreSQl, metabase and adminer which were totally new to be explored. Working with the real dataset with nearly 2.8 million rows and 370 columns let me understand why we need distributed tools like Spark. Without Spark, tasks will be processed very slow or impossible to load using the pandas library. It allows us to manage more complex tasks since it uses parallel processing and lazy evaluation. Example for lazy evaluation is there is no actual data processing occurring when calling .read() or select(). Processing only occurs when there are actions like .count() or .write(). This is actually why loading data is complete instantly while when Parquet writes, it takes about 5 minutes to execute.

The biggest challenge faced was setting everything up on Windows because the tutorial was designed for Linux. Many steps taught in the tutorial website did not work directly, I had to find alternative ways to do it. For example, the wget, rm is actually for Linux, it needs to be replaced with different Python Scripts using request or glob libraries. In addition, my PySpark 3.5.3 is incompatible with Python 3.14, requiring a redownload Python 3.11. There is also a problem I met which is the same with my teammate where the Metabase continually failed to start because of the compatibility issue with the PostgreSQL 9.6 container which was too outdated. However, these issues I met give me a deeper understanding of these tools and the setup behind that is linking them one to another.

The transformation phase deepened on the understanding of how pipelines actually work in place. Using PySpark, the raw-flat data was transformed into Star Schema consisting of 12 dimension tables and 1 fact table. Each dimension table was built by selecting relevant columns, removing duplicates and assigning a unique ID then writing them to PostgreSQL. An important lesson learned was that Spark can only write data into PostgreSQL but it cannot set any constraints like primary keys. A separate psycopg2 connection is needed to execute the alter table and add the primary key to each dimension table. I think these are the most memorizable things throughout this tutorial that make me truly know the answers behind these steps.

To be honest I was proud to be in this team as we have a great teamwork. We used to have a few discussions. Although our report is separate into persons for different parts, when we meet problems like I misunderstood the rubrics, we will have discussion together and try to understand and help each other without staying in doubt. Not only new tools with additional skills learned, we also know how important communication and teamwork are when doing group projects.

Lau Yee Wen

This tutorial was an eye-opening experience because knowledge about how a complete end-to-end ETL process can be developed using Apache Spark was gained. Before this project was started, no experience in using big data processing tools had been gained, especially for processing a dataset containing about 2.8 million rows and 370 fields. Through this experience, the importance of tools such as Apache Spark was understood.

The process of completing this tutorial was not completed smoothly, and several challenges were faced. During the first attempt, fake generated data was used to understand the process and become familiar with the tools. Although the data was not real, it was useful for understanding the overall structure of the ETL process, including data extraction, conversion from CSV to Parquet format, transformation into a star schema, loading into PostgreSQL, and final visualization in Metabase.

During the second attempt, the Brazilian school census data from 2010 to 2021 were correctly used according to the tutorial, and several challenges were encountered. For example, problems during the download process, coding issues, and Windows-specific configuration problems related to WinUtils and Hadoop had to be solved. From this experience, an important lesson was learned, which was to never give up. A deeper understanding of these tools was gained through every error that was faced, and more confidence was developed whenever a problem was solved. It was also understood that real data engineering work is not always completed successfully on the first attempt, and that both persistence and strong technical knowledge are important.

Overall, theory and practice were well connected through this tutorial, and a stronger understanding of big data processing, data modeling, and pipeline development was developed for future projects.

Poh Lok Yee

Throughout the project, it provides a deeper understanding of Apache Spark than theoretical knowledge alone. Seeing how Spark handles massive amounts of data with processing and converting 2.2GB of raw CSV data to Parquet format, creating the dimensions tables and loading 2.79 million rows data into PostgreSQL made it clear why Spark is widely used in the data engineering field. Not only the Spark, it also provided a clearer picture of how data moves within a real ETL pipeline system from extracting raw CSV files, transforming them using PySpark operations, and loading the structured data into a PostgreSQL database. For example, Spark cannot directly add primary keys to a PostgreSQL table and requires psycopg2 to execute the ALTER TABLE command separately. This showed how different tools connect and complement each other within the same pipeline.

The journey did not go smoothly . Several technical issues were faced throughout the project and most of them were setup environment problems. The wget command was incompatible with Windows and it required a PowerShell. The file encoding had to be changed from latin1 to iso-8859-1 as spark 4.1.1 version no longer supports the former. The biggest challenge was that Metabase continually failed to start due to the compatibility issue with the PostgreSQL 9.6 container, which was too outdated to be supported. These challenges were overcome through a combination of individual online research, consulting colleagues that worked on the same tutorial and using AI technologies to diagnose errors and identify suitable solutions. These troubleshooting processes also helped in understanding how to create precise AI prompts to get better and more accurate assistance when diagnosing complex system error.

Working as a group also brought its own lesson. Splitting the report by CRISP-DM phases allowed each member to develop deeper knowledge in their assigned section while maintaining good communication to ensure everything connected well as a whole. The team actively shared knowledge and taught each other throughout the tutorial. When one member resolved an error, the solution was shared to the rest of the group to help everyone gain a more complete understanding beside just their own section.

By the end of the tutorial, members not only had obtained new technical skills and a better understanding of actual data engineering workflow across tools such as Docker, Metabase, and PySpark, but had also enhanced essential soft skills, including technical problem-solving and effective teamwork.